

SECURE CODING REVIEW REPORT

CYBER SECURITY TASK 3 – CodeAlpha

Submitted By:

Name: THIRUKUMARAN S

Task Name: Secure Coding Review

Programming Language: PYTHON

Application Selected: Secure Coding Analysis

Date Of Submission: MAY 2026

Abstract:

Secure coding is an essential practice in software development that helps prevent security vulnerabilities and protects applications from potential cyber threats. This project focuses on conducting a secure coding review of a Flask-based web application. The review combines manual code inspection and automated static analysis to identify security weaknesses and assess the application's adherence to secure coding practices. The assessment evaluates authentication mechanisms, input validation, file upload security, session management, and error handling. Recommendations are provided to improve the overall security posture of the application.

Introduction:

Secure coding review is a process of examining source code to identify security flaws before deployment. By analyzing code during development, organizations can reduce vulnerabilities, prevent exploitation, and improve software reliability.

The objective of this task is to perform a security audit of a Python Flask web application and identify areas where security controls can be improved.

Objective:

The primary objectives of this review are:

- Analyze the source code of a Flask web application.
- Identify security weaknesses through manual inspection.
- Perform static code analysis using Bandit.
- Evaluate secure coding practices.
- Provide remediation recommendations.
- Document findings in a professional security assessment report.

Scope of Assessment:

The assessment focused on the following components of the application:

- Login functionality

- User input handling
- Comment submission feature
- File upload functionality
- Session management
- Error handling configuration
- Static code security analysis

The review was limited to source code inspection and did not involve penetration testing or exploitation attempts.

Tools Used:

Tool	Purpose
Kali Linux	Testing Environment
Python 3	Application Development
Flask	Web Framework
Bandit	Static Code Security Analyzer
Nano Editor	Source Code Review

Methodology:

The security assessment was conducted in two phases:

Manual Code Review:

The source code was manually inspected to identify:

- Authentication weaknesses
- Input validation issues
- File upload security concerns
- Session management deficiencies
- Error handling practices

Static Analysis:

Bandit was used to perform automated source code analysis.

Command Used:

bandit app.py

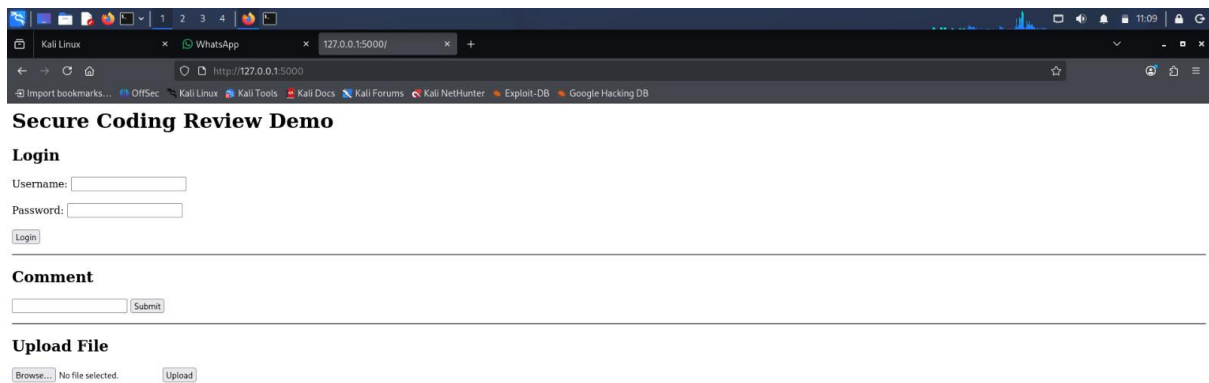
The scan reviewed the application for common Python security issues and coding mistakes.

Application Overview:

The Flask application provides the following functionalities:

1. User Login Form
2. Comment Submission Form
3. File Upload Feature

The application serves as a simple demonstration environment for secure coding review.

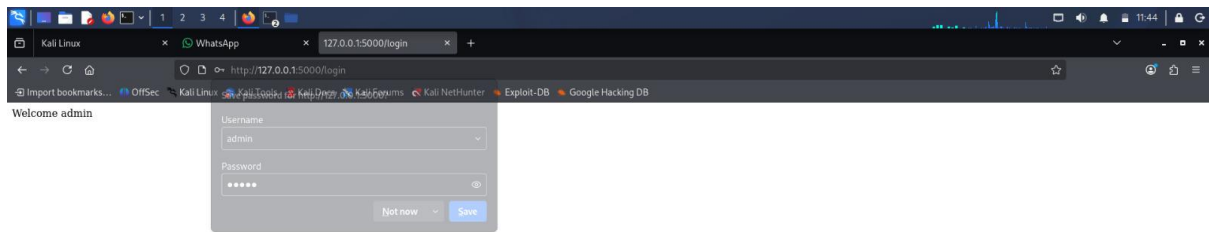


Security Analysis and Findings

Authentication Review

Code Reviewed

The login functionality accepts username input and displays a welcome message.



Observation:

The application does not implement a complete authentication mechanism. Password verification and password hashing are absent.

Risk:

Without password hashing and secure authentication controls, credentials may be exposed if authentication functionality is implemented incorrectly in the future.

Severity:

Medium

Recommendation:

- Implement password hashing using bcrypt or Argon2.
- Use secure authentication frameworks.
- Enforce strong password policies.
- Implement multi-factor authentication where applicable.

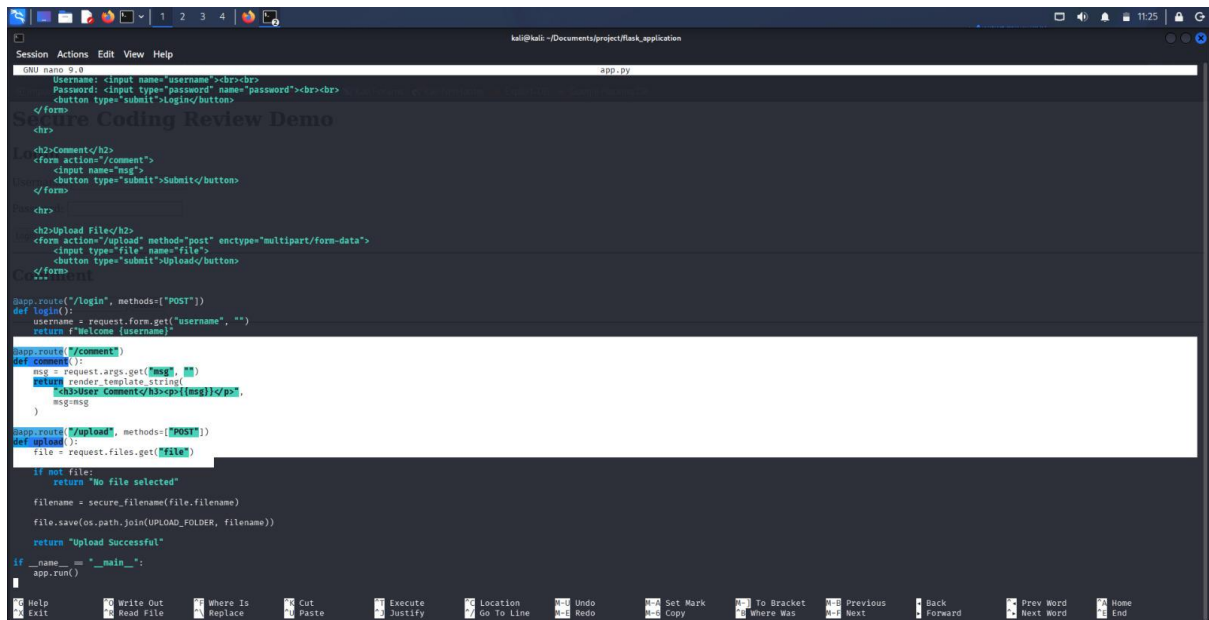
Input Validation Review:

Code Reviewed

The application accepts user input through:

`request.form.get()`

`request.args.get()`



The screenshot shows a web browser window displaying a web application interface. The interface includes a login form with fields for username and password, a comment form with a text area and a submit button, and an upload form with a file input and a submit button. The browser's address bar shows the URL `http://127.0.0.1:5000/`. Below the browser window, a terminal window displays the application's source code, which is a Python Flask application. The code defines routes for login, comment, and upload, and includes a main function to run the application.

```
#!/usr/bin/env python
from flask import Flask, request, redirect, render_template, url_for
import os

app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/login", methods=["POST"])
def login():
    username = request.form.get("username")
    password = request.form.get("password")
    return redirect(url_for("comment"))

@app.route("/comment", methods=["POST"])
def comment():
    msg = request.args.get("msg")
    return render_template("comment.html", msg=msg)

@app.route("/upload", methods=["POST"])
def upload():
    file = request.files.get("file")
    if not file:
        return "No file selected"
    filename = secure_filename(file.filename)
    file.save(os.path.join(UPLOAD_FOLDER, filename))
    return "Upload Successful"

if __name__ == "__main__":
    app.run()
```

Observation:

Input validation controls such as length restrictions, pattern matching, and sanitization are not implemented.

Risk:

Improper input validation can increase the likelihood of security issues such as injection attacks, malformed input processing, and application instability.

Severity:

Medium

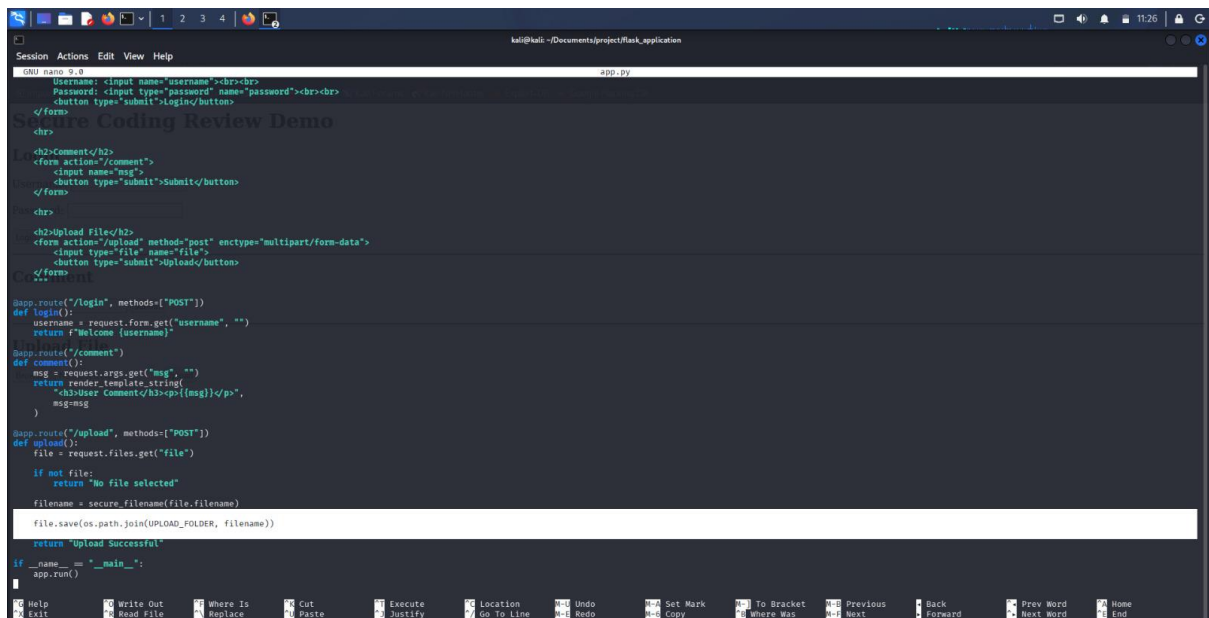
Recommendation:

- Validate all user inputs.
- Apply length restrictions.
- Use regular expressions where appropriate.
- Reject unexpected input values.

File Upload Security Review:

Code Reviewed

The upload functionality accepts files and stores them in the uploads directory.



```
#!/usr/bin/env python
# coding: utf-8

from flask import Flask, request, render_template, redirect, url_for
import os
import secure_filename

app = Flask(__name__)

UPLOAD_FOLDER = 'uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/login", methods=["POST"])
def login():
    username = request.form.get("username", "")
    password = request.form.get("password", "")
    return f"Welcome {username}"

@app.route("/comment", methods=["POST"])
def comment():
    msg = request.args.get("msg", "")
    return render_template_string(
        "<h3>User Comment</h3><p>{{msg}}</p>",
        msg=msg
    )

@app.route("/upload", methods=["POST"])
def upload():
    file = request.files.get("file")
    if not file:
        return "No file selected"
    filename = secure_filename(file.filename)
    file.save(os.path.join(UPLOAD_FOLDER, filename))
    return "Upload Successful"

if __name__ == "__main__":
    app.run()
```

Observation:

Although `secure_filename()` is used, there are no restrictions on:

- File type
- File extension
- File size

Risk:

Unrestricted file uploads may allow storage of unwanted or malicious files, resulting in increased security risk.

Severity:

Medium

Recommendation:

- Allow only approved file extensions.
- Restrict upload size.
- Validate MIME types.
- Scan uploaded files before storage.

Session Security Review:

Review Method

A manual search was conducted for Flask session handling mechanisms.

(Figure 5: Session Security Review)

Observation:

No session management implementation was identified.

Risk:

Without session management, user authentication states cannot be securely maintained.

Severity:

Medium

Recommendation:

- Implement Flask session management.
- Configure secure cookies.
- Enable HttpOnly and Secure flags.
- Apply session expiration controls.

Error Handling Review:

Code Reviewed

Application startup configuration:

```
app.run()
```


Observation:

Debug mode is not enabled.

Risk:

The risk of information disclosure through debugging output is minimized.

Severity:

Low

Recommendation:

Continue disabling debug mode in production environments.

Static Analysis Results:**Tool Used:**

Bandit

Command

bandit app.py

```
Session Actions Edit View Help
!$ pip install bandit --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Collecting bandit
  Downloading bandit-1.9.4-py3-none-any.whl.metadata (7.1 kB)
Requirement already satisfied: PyYAML>5.3.1 in /usr/lib/python3/dist-packages (from bandit) (6.0.3)
Collecting stevedore>2.20.0 (from bandit)
  Downloading stevedore-5.8.0-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: rich in /usr/lib/python3/dist-packages (from bandit) (13.9.4)
Requirement already satisfied: markdown-it-py>2.2.0 in /usr/lib/python3/dist-packages (from rich->bandit) (3.0.0)
Requirement already satisfied: pygments<3.0.0, >2.13.0 in /usr/lib/python3/dist-packages (from rich->bandit) (2.19.2)
Requirement already satisfied: mdurl<0.1 in /usr/lib/python3/dist-packages (from markdown-it-py>2.2.0->rich->bandit) (0.1.2)
Downloading bandit-1.9.4-py3-none-any.whl (134 kB)
Downloading stevedore-5.8.0-py3-none-any.whl (54 kB)
Installing collected packages: stevedore, bandit
Successfully installed bandit-1.9.4 stevedore-5.8.0

(kali@kali)~$ bandit --version
bandit 1.9.4
python version = 3.13.12 (main, Feb  4 2026, 15:06:39) [GCC 15.2.0]

(kali@kali)~$ cd Documents/project/flask_application
(kali@kali)~/Documents/project/flask_application$ bandit app.py
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.13.12
Run started:2026-06-01 05:17:43.098057+00:00

Test results:
No issues identified.

Code scanned:
Total lines of code: 50
Total lines skipped (#nosec): 0

Run metrics:
Total issues (by severity):
  Undefined: 0
  Low: 0
  Medium: 0
  High: 0
Total issues (by confidence):
  Undefined: 0
  Low: 0
  Medium: 0
  High: 0

Files skipped (.):

(kali@kali)~/Documents/project/flask_application$
```

Observation:

The Bandit scan completed successfully and reported no direct security issues.

Interpretation:

Automated analysis did not identify any critical vulnerabilities. Therefore, manual code review was essential for evaluating security controls and identifying areas for improvement.

Security Assessment Summary:

Security Area	Status	Observation
Authentication	Needs Improvement	Password hashing absent
Input Validation	Partial	Limited validation controls
File Upload Security	Needs Improvement	No file restrictions
Session Security	Not Implemented	Session management missing
Error Handling	Acceptable	Debug mode disabled
Static Analysis	Passed	No issues detected by Bandit

Recommendations:

1. Implement secure authentication mechanisms.
2. Hash passwords using bcrypt or Argon2.
3. Validate and sanitize all user input.

4. Restrict file upload types and file sizes.
5. Implement secure session management.
6. Enable secure cookie attributes.
7. Perform regular code reviews.
8. Integrate static analysis into the development lifecycle.
9. Maintain updated dependencies.
10. Follow OWASP Secure Coding Guidelines.

Conclusion:

A secure coding review was conducted on a Flask-based web application using both manual inspection and automated static analysis. While the Bandit scan did not identify direct security vulnerabilities, the manual review revealed several areas that require improvement, including authentication, input validation, file upload security, and session management. Implementing the recommended security controls will improve the application's resilience against potential threats and support the development of secure software systems.